

Robust AI Navigation in Distributed Environments: Balancing Believability and Precision

Dohun Lee

1 Introduction

In modern multiplayer games, Artificial Intelligence (AI) agents face a unique "split-brain" challenge. The "Logical Agent" resides on the server, where it has access to the Navigation Mesh (NavMesh) to calculate complex paths and execute steering behaviors. However, the player interacts with a "Visual Proxy" on the client, which receives only intermittent updates about the agent's state. When network conditions degrade, characterized by high latency or packet loss, this synchronization breaks down.

The result is often an AI that appears "broken." An agent that teleports or jitters fails the "Visual Turing Test," destroying player immersion regardless of how smart the server-side logic actually is. This article investigates a client-side prediction system designed to mask these network artifacts. We implement a hybrid Finite State Machine (FSM) combining Entity Interpolation and Dead Reckoning to bridge the gap between the server's truth and the client's reality.

The goal of this experiment is to quantify the trade-off between spatial accuracy (Precision) and movement fluidity (Believability). We analyze the agent's behavior under severe network stress (up to 310ms latency and 61% packet loss) to determine if client-side prediction can maintain the illusion of intelligence when the network fails.

2 System Architecture

The simulation is built on a decoupled Server-Client architecture using Unreal Engine. The system is designed to isolate the navigation logic from the movement execution, allowing us to measure the discrepancy between the two.

2.1 Server-Side Navigation (The Brain)

The "ANetworkManager" acts as the authoritative AI controller. It utilizes "UNavigationSystemV1" to query the NavMesh and generate A* paths.

- **Pathfinding:** The server calculates a path from the agent's current location to a random valid point within a 2000-unit radius.
- **Steering:** The agent moves at a fixed speed of 600 units/s. The server broadcasts "FEntitySnapshot" packets containing the Position, Velocity, and ServerTimestamp at every tick.

2.2 Client-Side Prediction (The Emulator)

The client is a "dumb" terminal with no knowledge of the NavMesh. Its sole responsibility is to render the agent smoothly based on the SnapshotBuffer. The client uses a State Machine to decide how to render the agent:

- **State 0 (OFF):** Raw synchronization. The actor snaps immediately to the latest packet position.
- **State 1 (Interpolation):** The client renders the agent in the past (delayed by 0.7s), blending between two known snapshots to smooth out jitter.
- **State 2 (Dead Reckoning):** When packet loss creates a gap larger than 0.5s, the client predicts the agent's future position using its last known velocity.

3 Client-Side Prediction Logic

The core of the client's "Movement AI" is the prediction logic. The client attempts to act as a smart emulator, filling in the gaps left by the network.

3.1 Linear Interpolation (Smoothing)

The client first attempts to find two snapshots, S_{start} and S_{end} , that bracket the current render time. The position is calculated using linear interpolation as defined in Equation 1.

$$P(t) = P_{start} + \alpha(P_{end} - P_{start}) \quad (1)$$

Where α is the normalized time factor calculated as:

$$\alpha = \frac{t_{render} - t_{start}}{t_{end} - t_{start}} \quad (2)$$

Listing 1 shows the implementation of this logic, ensuring smooth transitions between network updates.

Listing 1 Linear interpolation implementation in the client loop.

```
// Source: NetworkAgent.cpp
float TimeGap = End.ServerTimestamp - Start.ServerTimestamp;
if (TimeGap > KINDA_SMALL_NUMBER) { float Alpha = (RenderTime
- Start.ServerTimestamp) / TimeGap;

// Linear Interpolation (Lerp)
NewLocation = FMath::Lerp(Start.Position, End.Position,
```

```
Alpha);
NewVelocity = FMath::Lerp(Start.Velocity, End.Velocity,
Alpha);
```

3.2 Dead Reckoning (Prediction)

When packet loss occurs, the buffer empties. The client switches to Dead Reckoning, predicting the future position based on the last known velocity. This is modeled by the equation of motion shown in Equation 3.

$$P_{new} = P_{last} + V_{last} \times \Delta t \quad (3)$$

This linear prediction allows the agent to continue moving even without server updates, as shown in Listing 2.

Listing 2 **Dead Reckoning fallback logic.**

```
// Source: NetworkAgent.cpp
if (!bFoundInterpolation && DRSnapshot.ServerTimestamp >
0.0f)
{
    // State 2: Dead Reckoning
    // Assumption: The AI agent will not change velocity
(Newtonian prediction)
    bIsDeadReckoning = true;
    float TimeSinceLast = RenderTime -
DRSnapshot.ServerTimestamp;

    // Extrapolate position based on last known velocity
(Steering)
    // Limitation: Cannot predict turns without NavMesh
    FVector PredictedPos = DRSnapshot.Position +
(DRSnapshot.Velocity * TimeSinceLast);

    NewLocation = PredictedPos;
}
```

This logic represents a fundamental assumption: "The agent will continue doing what it was doing." While this holds true for linear motion, we will see in Section 4 how this fails during complex steering maneuvers.

4 Experimental Analysis

We subjected the AI to four distinct network scenarios to evaluate its robustness. Please refer to Table 1 for the specific parameters used in the stress tests.

Table 1 Summary of experimental scenarios defined by latency and packet loss rates.

Scenario	Latency	Packet Loss	Condition Description
Lat110_Loss20	110ms	20%	Moderate latency, Moderate loss
Lat110_Loss61	110ms	61%	Moderate latency, Extreme loss
Lat304_Loss0	304ms	0%	High latency, No loss
Lat310_Loss40	310ms	40%	High latency, High loss

4.1 The Trade-off: Believability vs. Precision

The most significant finding is the inverse relationship between spatial accuracy and visual smoothness. Please refer to Figure 1.

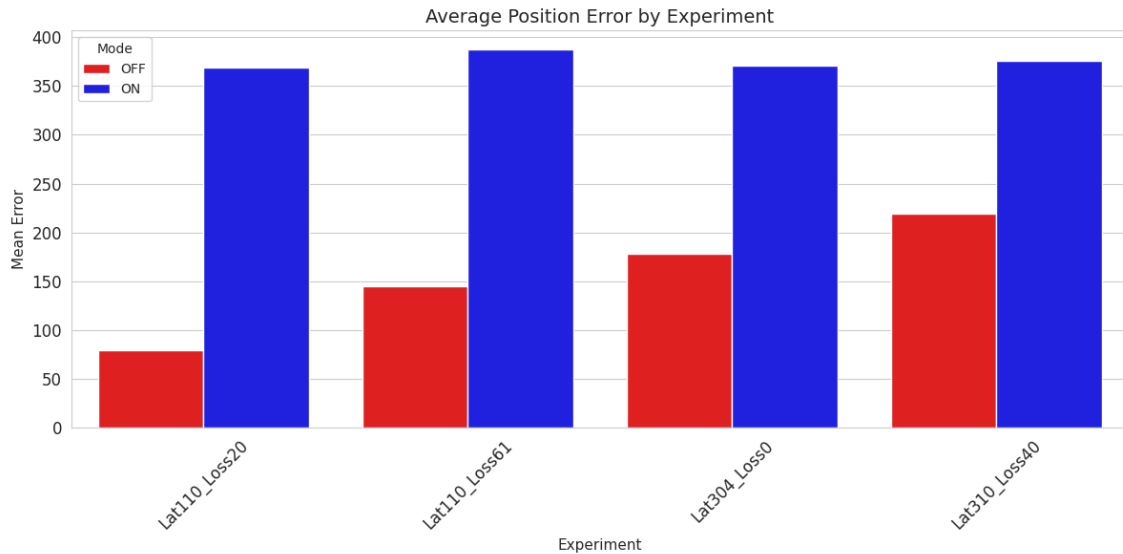


Figure 1 The "OFF" mode (Red) has low error but high jerkiness, while "ON" (Blue) sacrifices accuracy for smoothness.

As shown in Figure 1, the "OFF" state (Red) maintains lower error but suffers from massive acceleration spikes (10^5 magnitude). This creates a "teleporting" effect. The "ON" state (Blue) flattens the acceleration curve but maintains a consistent error of ~ 380 units due to the fixed interpolation delay.

Figure 2 highlights the "Believability" gain. The prediction algorithm reduces the positional jerkiness by orders of magnitude, transforming the movement from a glitchy mess into a fluid path.

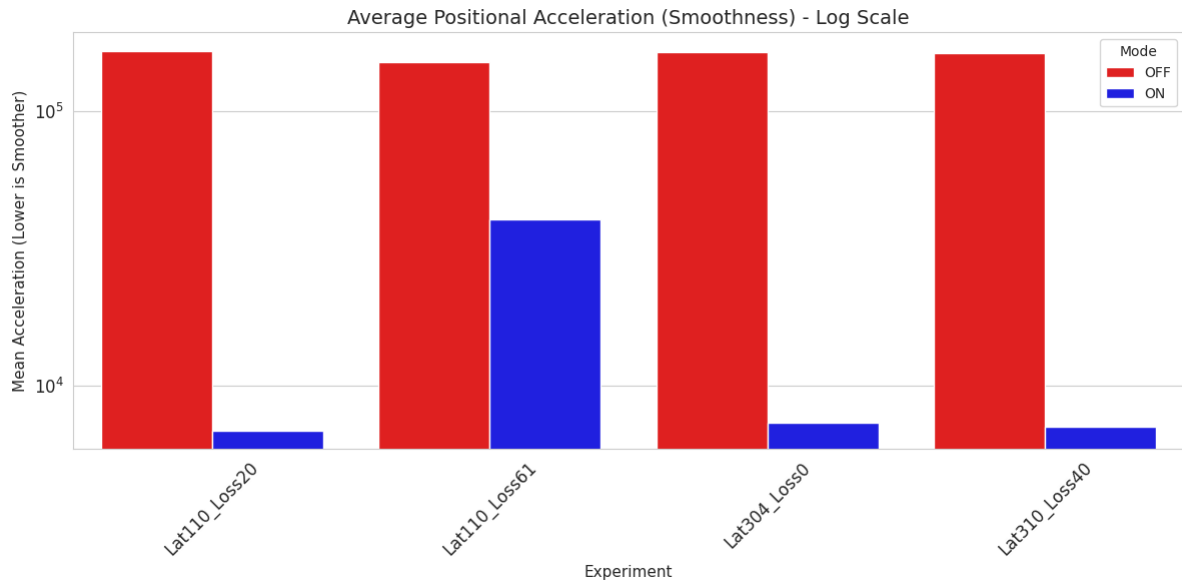


Figure 2 Average positional acceleration (jerkiness) comparison on a log scale.

4.2 Coping with Packet Loss (The Overshooting Problem)

In the Lat110_Loss61 scenario, over half of the server updates were lost. The "OFF" client became erratic, rendering the path unintelligible. The "ON" client successfully engaged Dead Reckoning to maintain movement.

However, this revealed a critical flaw in client-side AI. Please refer to Figure 3.

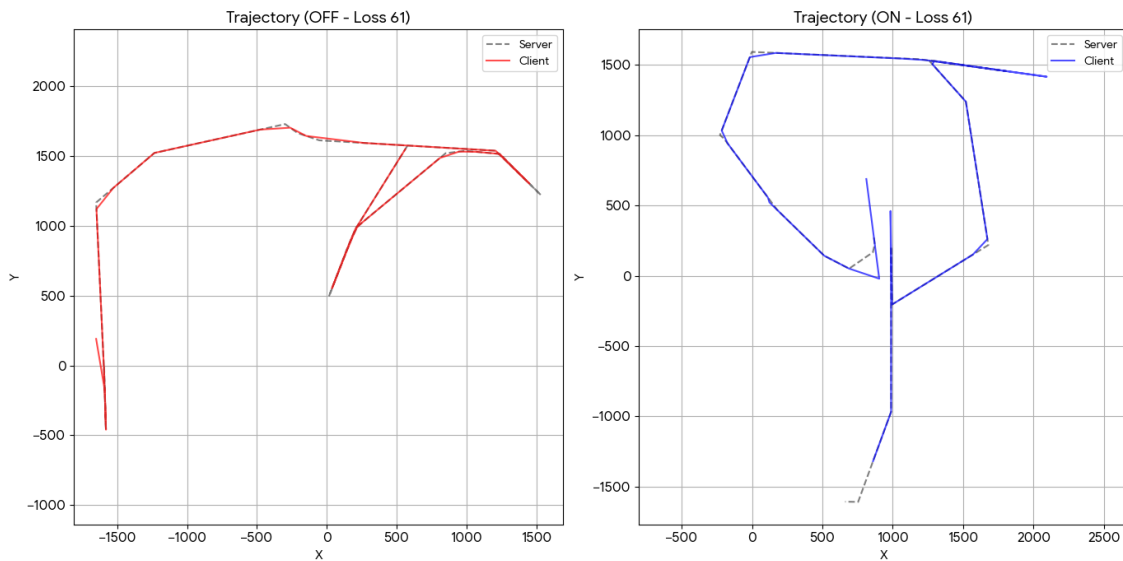


Figure 3 Trajectory under 61% packet loss showing "overshooting" deviations in the ON state (Right).

The blue line (Client) sharply deviates from the grey dashed line (Server) at corners. This occurs because the client uses Linear Dead Reckoning. When the Server AI turns a corner on the NavMesh during a packet loss window, the Client AI continues straight because it lacks local NavMesh awareness. This "Overshooting" is the cost of blind prediction.

Figure 4 confirms that these error spikes coincide directly with the algorithm switching to State 2 (Dead Reckoning).



Figure 4 High position error peaks align with the activation of State 2 (Dead Reckoning).

4.3 High Latency Ghosting

At 310ms latency (Loss 40%), the algorithm rarely engaged Dead Reckoning, remaining mostly in State 1 (Interpolation). The large buffer (0.7s) absorbed the gaps, creating a smooth "Ghost" effect. Please refer to Figure 5.

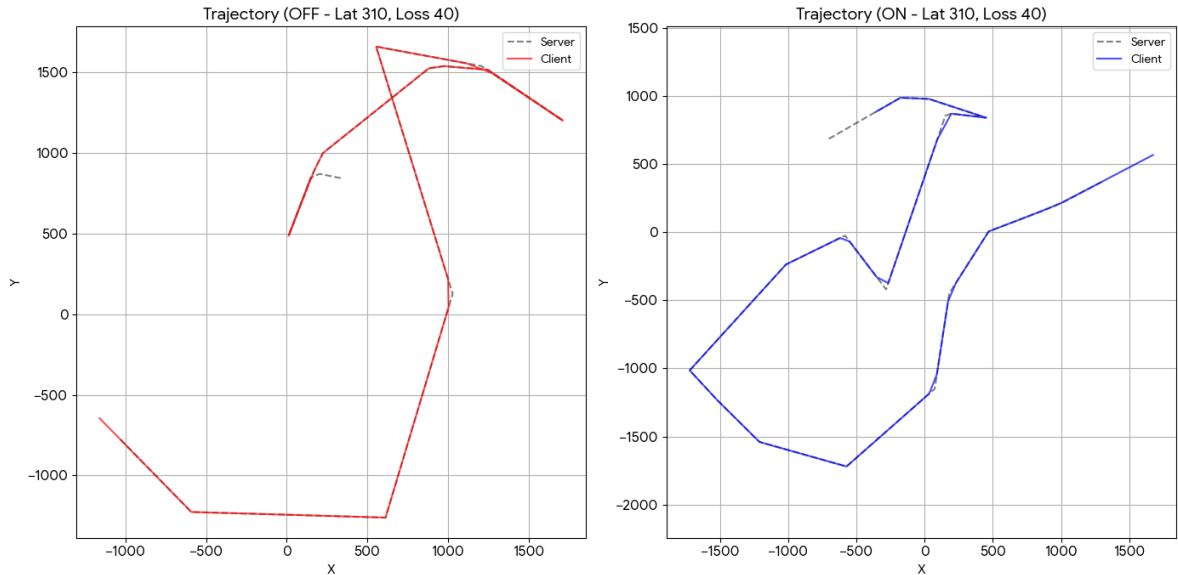


Figure 5 The Client path (Blue) perfectly mimics the Server path shape (Grey) but is shifted in time.

While visually perfect, this "Ghosting" introduces a gameplay challenge. The player sees the AI at a position it occupied 700ms ago. In a shooter game, this would require server-side "Lag Compensation" (rewinding time) to ensure shots land where the player aimed.

Figure 6 illustrates the success of the smoothing. Even at 310ms latency, the acceleration profile remains flat and human-like.

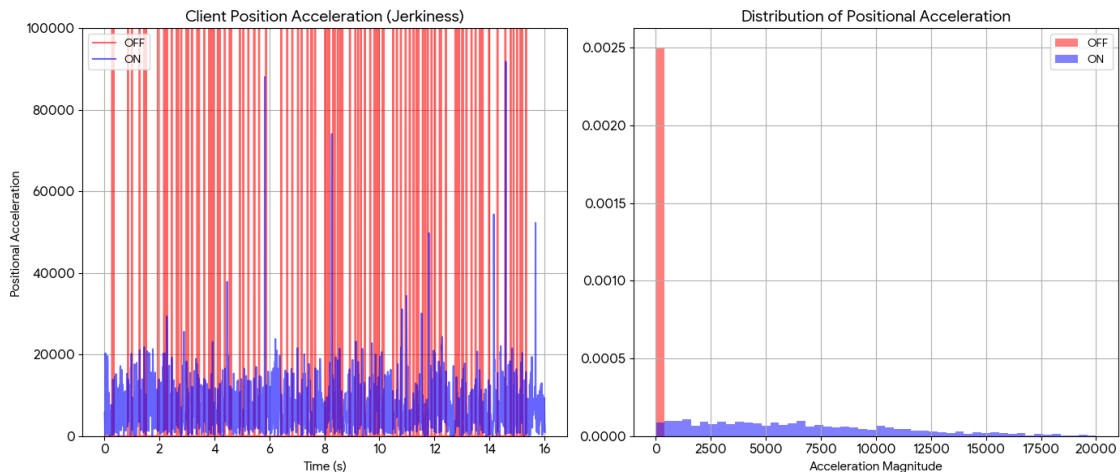


Figure 6 The "ON" state (Blue) maintains human-like acceleration limits even under high stress.

5 Conclusion

This experiment demonstrates that client-side compensation is mandatory for networked AI to appear intelligent. Raw synchronization results in acceleration values physically impossible for a walking character, whereas the compensated motion falls within believable limits.

However, the "Overshooting" phenomenon highlights that Linear Dead Reckoning is insufficient for complex AI navigation. Future work should focus on NavMesh-Aware Prediction. By giving the client a simplified local NavMesh, the "Visual Proxy" could run a local pathfinding query during packet loss, allowing it to predict turns and avoid walls, thus maintaining behavioral fidelity even when disconnected.

9 References

Bernier, Y. 2001. [Latency Compensating Methods in Client/Server In-game Protocol Design and Optimization](#). Valve Developer Community.

Epic Games. Networked Movement in the Character Movement Component. Unreal Engine Documentation.

<https://dev.epicgames.com/documentation/en-us/unreal-engine/understanding-networked-movement-in-the-character-movement-component-for-unreal-engine>

Pantel, L. and Wolf, L.C. 2002. [On the suitability of dead reckoning schemes for games](#). Proceedings of the 1st Workshop on Network and System Support for Games